

LECTURE SLIDE EXCERPTS

VAEs, latent diffusion, and flow matching

Variational autoencoders and generative dynamics

Selected original slides from MIT course 6.S184

Peter Holderrieth and Ron Shprints

Course sponsor: Tommi Jaakkola

[Course slides and recordings](#)

The Need for Latent Spaces

- High-resolution image: 3 color channels, height 600, width 1000 $\rightarrow$ the resulting dimension is: $3 \cdot 600 \cdot 1000 = \mathbf{1.8 \text{ million!}}$
- **This is a very high-dimensional space!!!!**

Several problems:

- GPU memory blows up
- Learning problem is very hard
- Redundancy: Closeby pixels highly correlated

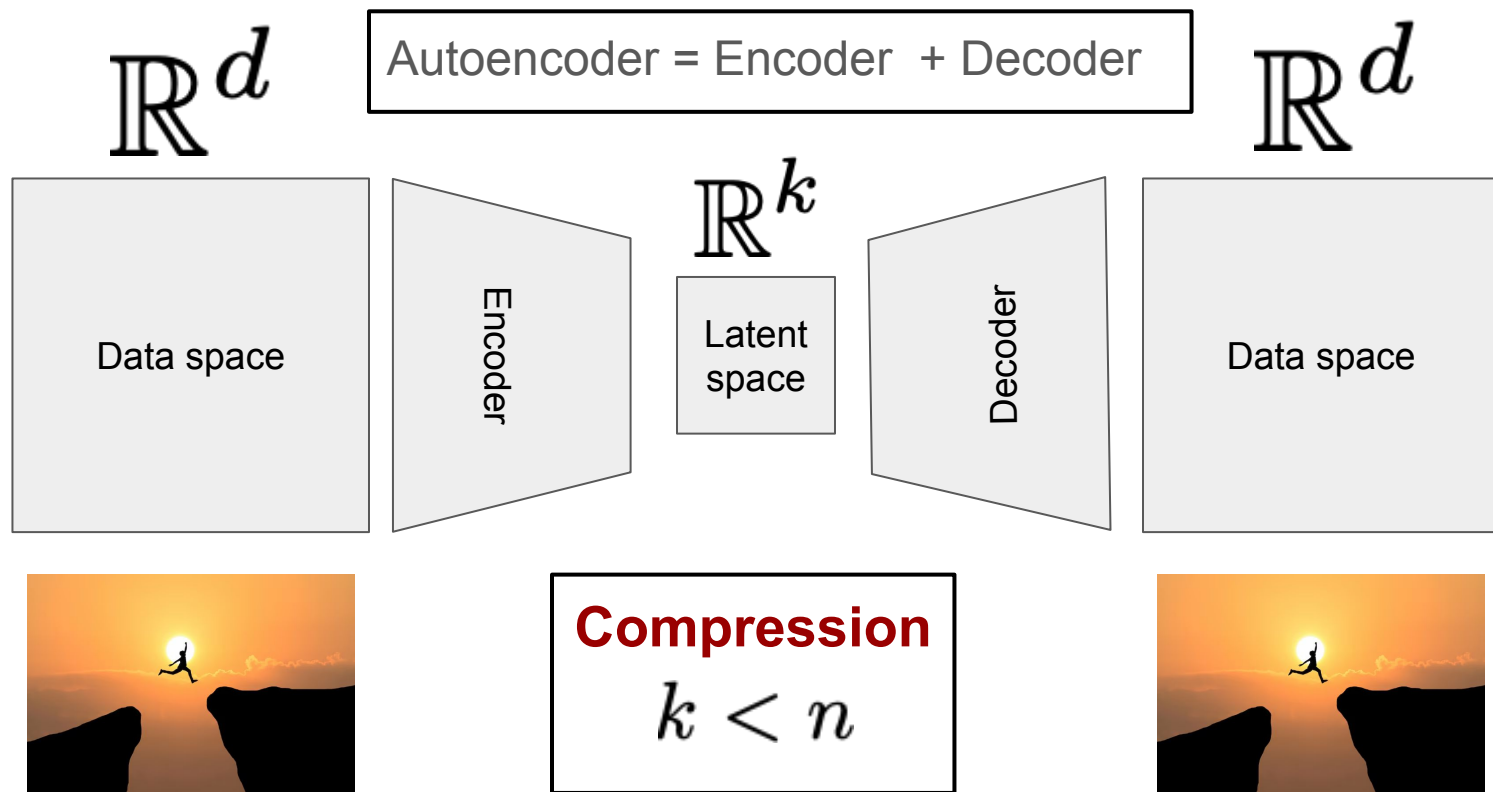


Why is this a problem for diffusion models and not for supervised learning?

$$u_t^\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$$

We learn a vector field - output is very high-dimensional!
We apply vector field many times (simulation of ODE)!

Latent Spaces are constructed via autoencoders



The Problem of Standard Autoencoders

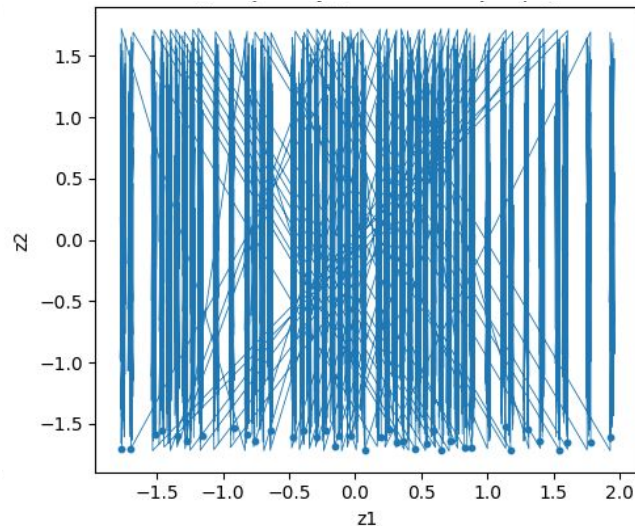
Our goal is to transform data into latents and then learn the distribution of latents.

Question: What happens to the data distribution when we transform it into latent space?

Answer: We don't know. The reconstruction loss does not say anything about the distribution.

Problem: We might make the learning problem (i.e. training) much harder. Therefore, we might be able to compress but not able to learn how to sample the distribution anymore.

A “bad” latent space



Goal: Create autoencoder that creates “nice” latent distribution

KL-divergence

For two probability densities q, p , the **Kullback-Leibler divergence** (KL-divergence) is defined as

$$D_{\text{KL}}(q(x) \parallel p(x)) = \int q(x) \log \frac{q(x)}{p(x)} = \mathbb{E}_{X \sim q} \left[\log \frac{q(X)}{p(X)} \right].$$

The KL divergences measures how different two distributions are. It fulfills the natural properties:

$$D_{\text{KL}}(q(x) \parallel p(x)) \geq 0,$$

$$D_{\text{KL}}(q(x) \parallel p(x)) = 0 \quad \Leftrightarrow \quad q = p.$$

VAE - Training Algorithm

Algorithm 7 β -VAE Training Procedure

Require: Dataset of samples $x \sim p_{\text{data}}$, encoder networks $(\mu_\phi(x), \log \sigma_\phi^2(x))$, decoder network $\mu_\theta(z)$, latent dim k , constants $\beta \geq 0$, $\sigma^2 > 0$

- 1: **for** each mini-batch $\{x_i\}_{i=1}^B$ **do**
- 2: Encode each x_i : $\mu_i \leftarrow \mu_\phi(x_i)$, $\log \sigma_i^2 \leftarrow \log \sigma_\phi^2(x_i)$
- 3: Sample noise $\epsilon_i \sim \mathcal{N}(0, I_k)$
- 4: Reparameterize: $z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$
- 5: Decode mean: $\hat{x}_i \leftarrow \mu_\theta(z_i)$
- 6: Reconstruction loss:

$$\mathcal{L}_{\text{recon}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2\sigma^2} \|x_i - \hat{x}_i\|^2$$

- 7: KL loss to the prior $p_{\text{prior}}(z) = \mathcal{N}(0, I_k)$:

$$\mathcal{L}_{\text{KL}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2} \sum_{j=1}^k (\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1)$$

- 8: Total loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$
 - 9: Update $(\phi, \theta) \leftarrow \text{grad_update}(\mathcal{L})$
 - 10: **end for**
-

2. LATENT DIFFUSION

Latent diffusion

The autoencoder supplies a compact representation.

The generative model learns the distribution of those latents.

The decoder maps sampled latents back to images.

Latent Diffusion Models (LDMs) - Recipe

1. **Data:** Take all training data $x_1, \dots, x_N$ (e.g. all images on the internet)
2. **Encoding:** Convert all images into latents (for VAE simply take the mean prediction)
3. **Latent data:** This gives us a dataset of latents $z_1, \dots, z_N$ of significantly smaller size than high resolution images
4. **Latent diffusion model:** Build a diffusion model for the dataset of latents, i.e. the diffusion model now generates latent vectors!
5. **Decoding:** After sampling from the diffusion model, map generated latent back to data space

Return decoded image as sample!

Note: Exact same recipe as before. Just with a transformed dataset!

3. FLOW MATCHING

Flow matching

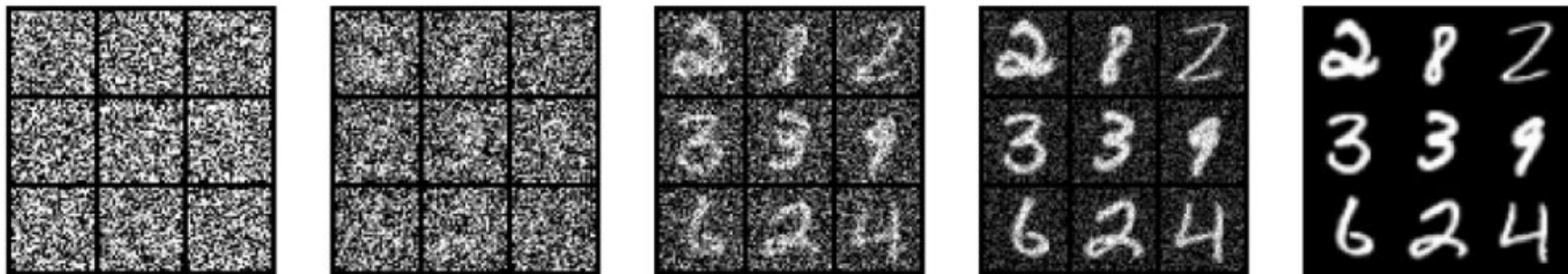
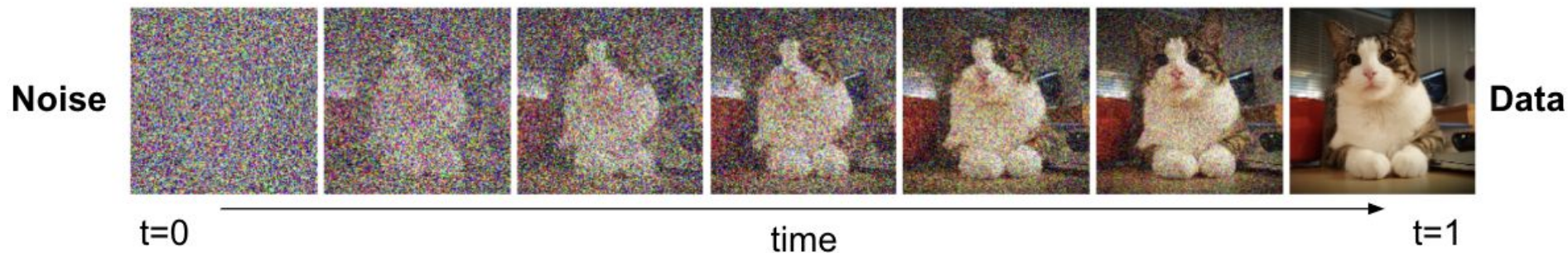
Notation in the following MIT slides:

z denotes a clean training example, x an intermediate state.

For an LDM, the training examples are encoded latents.

Time runs from noise at $t = 0$ to data at $t = 1$.

Probability Paths: The Path from Noise to Data



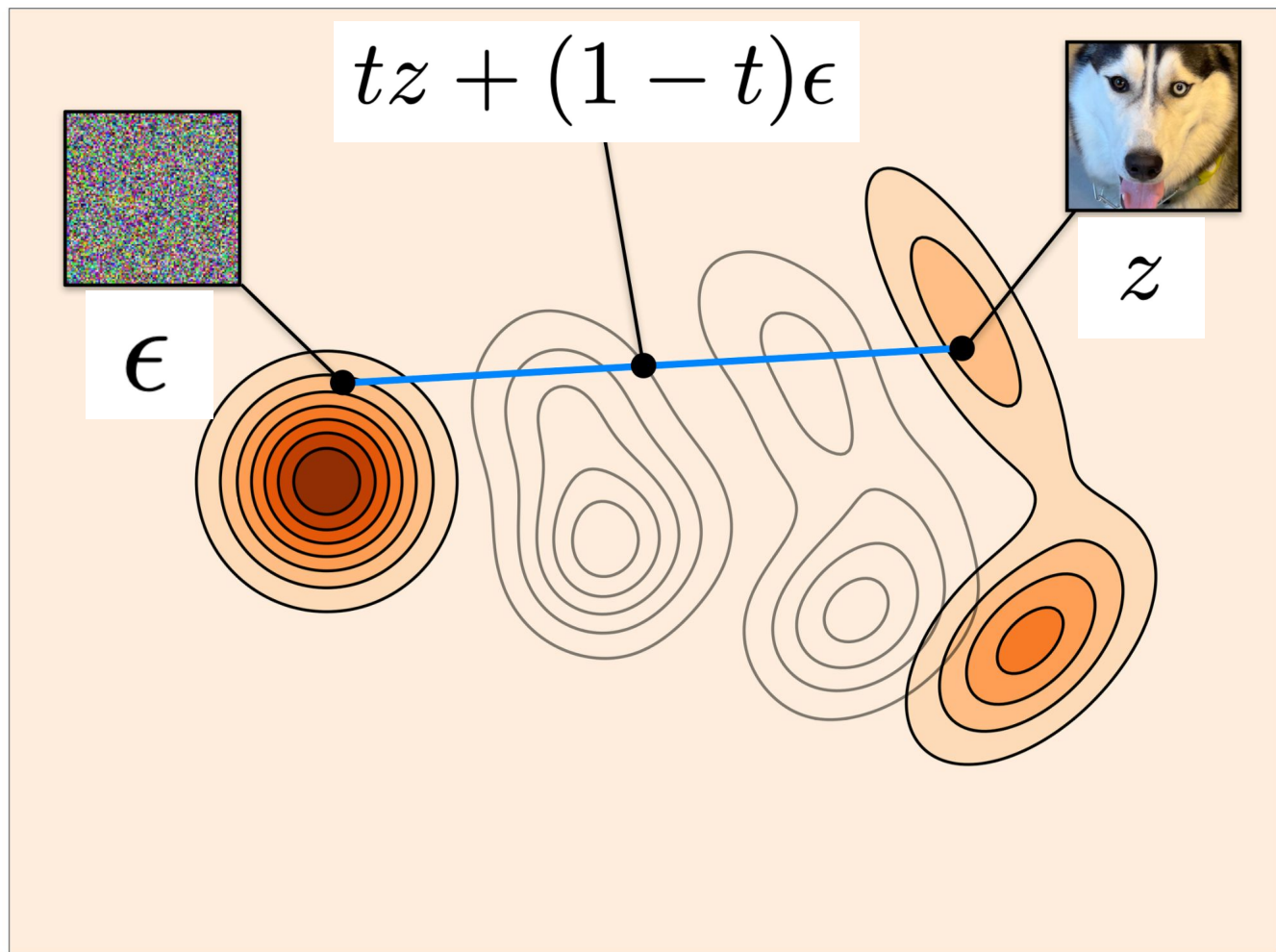


Figure
credit:
Yaron
Lipman